



1. Introduction to Java

Write Once,
Run Anywhere



1. What is Java ?

- Java is a high-level, class-based, object-oriented programming language.
- It is designed to have as few implementation dependencies as possible.
- Java is platform independent, secure, robust and portable.

3. James Gosling

- James Gosling is known as the "Father of Java".
- He designed Java with the goal of creating a language that is simple, secure and portable.

4. Why Java ?

- ✓ Simple and easy to learn
- ✓ Object-Oriented
- ✓ Platform Independent (Write Once, Run Anywhere)
- ✓ Secure and reliable
- ✓ Robust (Strong memory management)
- ✓ Multithreaded
- ✓ High performance
- ✓ Widely used in industries

6. Java vs Other Languages (Quick View)

Feature	Java	Other Languages (C/C++)
Platform	Independent	Dependent
Memory Management	Automatic (Garbage Collection)	Manual
Security	High	Low
Syntax	Simple	Complex
Portability	High	Low

8. How Java Program Works ?

2. History of Java

- Java was developed by James Gosling and his team at Sun Microsystems.
- Project started in 1991, named "Green".
- Officially released in 1995.
- Sun Microsystems was acquired by Oracle in 2010.

★ Fun Fact

Java was originally designed for small devices like set-top boxes, TV and appliances. 😊

5. Applications of Java

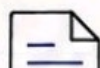
- Desktop Applications (e.g., IDEs, Tools)
- Web Applications (e.g., Banking, ERP)
- Android App Development
- Enterprise Applications
- Scientific Applications
- Big Data Technologies
- Embedded Systems
- Game Development

7. First Java Program

```
class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
} // Hello World Program
```



Compiler



JVM

Hello World!



2. JDK, JRE & JVM

★ Remember

JDK $\supset$ JRE $\supset$ JVM



1. What is JVM ?

- JVM (Java Virtual Machine) is the heart of Java.
- It is a runtime environment that executes Java bytecode.
- It makes Java platform independent.

2. What is JRE ?

- JRE (Java Runtime Environment) provides the libraries, JVM and other files required to run a Java program.
- It does not contain development tools (compiler, debugger etc.).

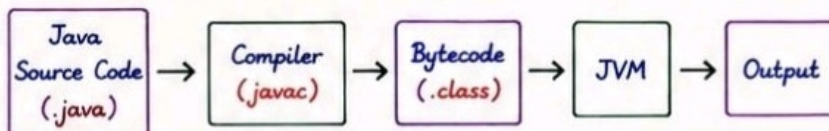
3. What is JDK ?

- JDK (Java Development Kit) is a complete package for development.
- It contains JRE + development tools like compiler (javac), debugger, javadoc, jar etc.
- It is used to create, compile and run Java programs.

4. JDK vs JRE vs JVM

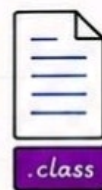
Feature	JDK	JRE	JVM
Full Form	Java Development Kit	Java Runtime Environment	Java Virtual Machine
Contains	JRE + Development Tools	JVM + Libraries + Other files	Runtime Engine
Used For	Development + Execution	Execution (Run)	Execution of Bytecode
Includes	javac, jar, javadoc, debugger etc.	JVM + Core Libraries + Resources	Class Loader, Runtime Data Areas, etc.
Size	Largest	Medium	Smallest

5. Java Compilation Process



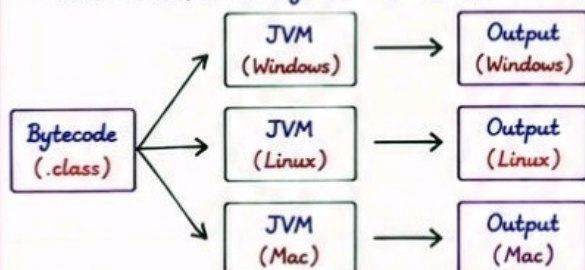
6. Bytecode

- When we compile a Java program, it is converted into bytecode (.class file).
- Bytecode is platform independent and can run on any system with JVM.
- Bytecode is not machine code.

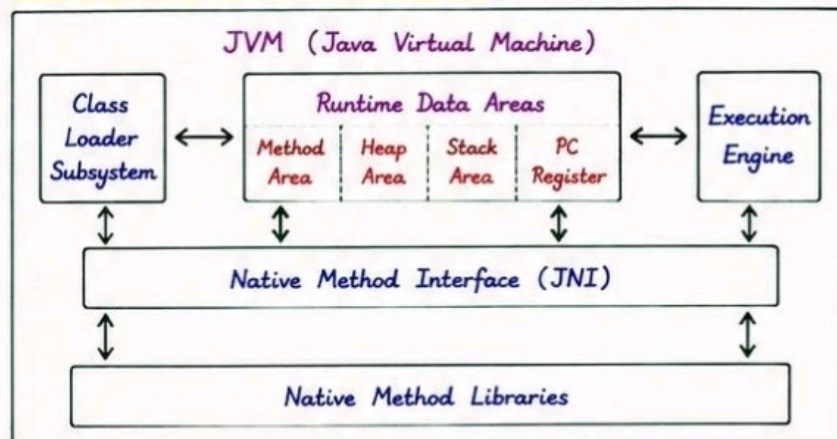


7. Platform Independence

- Java programs are compiled into bytecode.
- Bytecode can run on any OS where JVM is available.
- Write Once, Run Anywhere (WORA).



8. JVM Architecture (Overview)



JDK is for Developers, JRE is for Running, JVM is for Executing Bytecode.





3. Java Basics : Syntax, Variables & Data Types

★ Key Point

Understand the basics clearly, build strong foundation! 😊

1. Basic Java Program Structure

```
// My First Java Program
class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

- **class** : Blueprint or template.
- **HelloWorld** : Class name (Must start with capital letter).
- **main()** : Entry point of every Java program.
- **System.out.println()** : Prints output on console.
- Every statement ends with a semicolon (;)

3. Primitive Data Types

Type	Size	Range	Default Value	Example
byte	1 byte	-128 to 127	0	byte b = 10;
short	2 bytes	-32,768 to 32,767	0	short s = 1000;
int	4 bytes	-2^{31} to $2^{31}-1$	0	int i = 12345;
long	8 bytes	-2^{63} to $2^{63}-1$	0L	long l = 123456789L;
float	4 bytes	~6-7 digits	0.0f	float f = 5.6f;
double	8 bytes	~15 digits	0.0d	double d = 45.678;
char	2 bytes	0 to 65535	'\u0000'	char ch = 'A';
boolean	1 bit	true / false	false	boolean flag = true;

5. Type Casting

- Converting one data type into another.
- Two types : Implicit (Widening) and Explicit (Narrowing)

Implicit (Widening)

Smaller → Larger

byte → short → int → long
→ float → double

Ex.

int a = 10;
double b = a; // int to double

Explicit (Narrowing)

Larger → Smaller (cast needed)

double → float → long → int
→ short → byte

Ex.

double x = 25.7;
int y = (int) x; // double to int

2. Variables

- Variable is a name given to a memory location to store data.
- It can change its value during program execution.

Declaration and Initialization

```
int age; // declaration
age = 20; // initialization
double salary = 25000.50; // declaration + initialization
```

Rules for Variables

1. Must start with letter, _ or \$
2. Cannot start with a digit
3. No special characters allowed (except _ or \$)
4. Case sensitive (age, Age and AGE are different) 😊

4. Reference Data Types

- These are not primitive.
- They store reference (address) to objects.
- Examples :

String, Arrays, Classes, Interfaces, etc.

Example :

```
String name = "Abhi"; // Reference to String object
int[] marks = {90, 80, 70}; // Reference to array
```

6. Primitive vs Reference Types

Primitive Types	Reference Types
Store actual value	Store reference (address)
Fixed size	Variable size
Stored in Stack	Stored in Heap
Faster and efficient	Relatively slower
Example : int, char, float	Example : String, Arrays, Objects

★ Practice every syntax. Build strong basics, become Java Expert! 😊



4. Operators & Input / Output in Java

★ Key Point

Operators help us perform operations and make decisions in programs.



1. Arithmetic Operators

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus (Remainder)

Example :

```
int a = 10, b = 3;
a + b = 13
a - b = 7
a * b = 30
a / b = 3
a % b = 1
```

4. Assignment Operators

=	a = 10
+=	a += 5 (a = a + 5)
-=	a -= 3 (a = a - 3)
*=	a *= 2 (a = a * 2)
/=	a /= 2 (a = a / 2)
%=	a %= 2 (a = a % 2)

2. Relational Operators

==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example :

```
int a = 10, b = 20;
a == b → false
a != b → true
a > b → false
a < b → true
a >= b → false
a <= b → true
```

5. Unary Operators

+	Unary Plus
-	Unary Minus
++	Increment
--	Decrement
!	Logical NOT

Example :

```
int a = 5;
+a → 5
-a → -5
a++ → 5 (then a = 6)
a-- → 5 (then a = 4)
!true → false
```

3. Logical Operators

&&	Logical AND
	Logical OR
!	Logical NOT

Example :

```
boolean x = true, y = false;
x && y → false
x || y → true
!x → false
```

6. Ternary Operator

Syntax : condition ? expression1 : expression2;

Example :

```
int max = (a > b) ? a : b;
// If a > b, max = a else max = b
```

7. System.out.println()

- Used to print output on console.

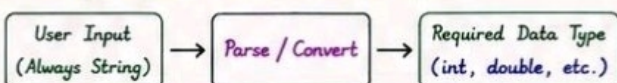
```
System.out.println("Hello Java!");
```

8. Scanner Class (User Input)

- Scanner is a class in java.util package.
- Used to take input from user.
- Create object first, then use methods.

```
Scanner sc = new Scanner(System.in);
```

10. Type Conversion During Input



Example :

- String s = "123";
- int n = Integer.parseInt(s); // String to int
- double d = Double.parseDouble(s); // String to double

9. Taking Input Examples

```
Scanner sc = new Scanner(System.in);
// int input
System.out.print("Enter an integer: ");
int n = sc.nextInt();
// double input
System.out.print("Enter a double value: ");
double d = sc.nextDouble();
// String input
System.out.print("Enter your name: ");
String name = sc.nextLine();
```

Output Example

Enter an integer:
25

Enter a double value:
12.75

Enter your name:
Abhi

★ Interview Tip

== vs .equals()

- == → compares memory (reference)
- .equals() → compares content (value)

Example :

```
String a = new String("hello");
String b = new String("hello");
a == b → false
a.equals(b) → true
```



★ Practice these operators daily. They are the building blocks of every Java program! 😊



5.

Conditional Statements & Loops in Java

★ Key Point

Control the Flow of Execution in a Program



1. If Statement

```
if (condition) {
    // code to be executed
}
```

Example :

```
int age = 18;
if (age >= 18) {
    System.out.println("You are eligible");
}
```

2. If-Else Statement

```
if (condition) {
    // code
} else {
    // code
}
```

Example :

```
int age = 16;
if (age >= 18)
    System.out.println("Adult");
else
    System.out.println("Minor");
```

3. Else-If Ladder

```
if (condition1) {
    // code
} else if (condition2) {
    // code
} else if (condition3) {
    // code
} else {
    // code
}
```

Example :

```
int marks = 72;
if (marks >= 90)
    System.out.println("A Grade");
else if (marks >= 75)
    System.out.println("B Grade");
else if (marks >= 50)
    System.out.println("C Grade");
else
    System.out.println("Fail");
```

4. Nested If

```
if (condition1) {
    if (condition2) {
        // code
    } else {
        // code
    }
}
```

Example :

```
int age = 20;
int weight = 70;
if (age >= 18) {
    if (weight >= 50)
        System.out.println("Eligible");
    else
        System.out.println("Weight Low");
} else {
    System.out.println("Not Eligible");
}
```

5. Switch Statement

```
switch (expression) {
    case value1:
        // code
        break;
    case value2:
        // code
        break;
    default:
        // code
}
```

Example :

```
int day = 3;
switch (day) {
    case 1: System.out.println("Mon"); break;
    case 2: System.out.println("Tue"); break;
    case 3: System.out.println("Wed"); break;
    case 4: System.out.println("Thu"); break;
    default: System.out.println("Invalid");
}
```

6. For Loop

```
for (initialization; condition; update) {
    // code
}
```

Example :

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

Output :

```
1
2
3
4
5
```

7. While Loop

```
while (condition) {
    // code
    // update
}
```

Example :

```
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
```

Output :

```
1
2
3
4
5
```

8. Do-While Loop

```
do {
    // code
    // update
} while (condition);
```

Example :

```
int i = 1;
do {
    System.out.println(i);
    i++;
} while (i <= 5);
```

Output :

```
1
2
3
4
5
```

9. Break Statement

→ Used to exit a loop or switch immediately.

Example :

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) break;
    System.out.println(i);
}
```

Output :

```
1
2
```

10. Continue Statement

→ Skips the current iteration and moves to next.

Example :

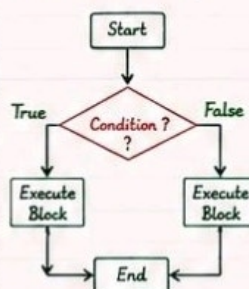
```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    System.out.println(i);
}
```

Output :

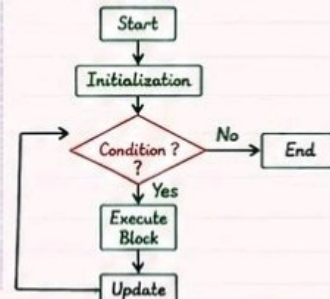
```
1
2
4
5
```

Flowcharts

A. Decision Making



B. Looping



Practice control flow. Master logic. Write better programs !





6. Methods in Java

★ Key Point

Methods help us break big problems into small, reusable pieces of code.



1. What is a Method?

- A method is a block of code that performs a specific task.
- It improves code reusability and readability.
- It is also called a function.

2. Method Declaration (Syntax)

```
accessModifier returnType methodName(parameterList) {
    // body of the method
    return value; // if returnType is not void
}
```

Example :

```
public int add(int a, int b) {
    int sum = a + b;
    return sum;
}
```

3. Calling a Method

- Create an object (for non-static methods).
- Call the method using object reference.
- For static methods, call using class name.

Non-static method call	Static method call
<pre>MyClass obj = new MyClass(); obj.myMethod();</pre>	<pre>MyClass.myStaticMethod();</pre>

5. Return Type

- A method can return a value to the caller.
- If no value is returned, use void.

With return	Without return
<pre>int square(int n) { return n * n; } int s = square(4); // 16</pre>	<pre>void greet() { System.out.println("Hello"); } greet();</pre>

7. Method Overloading

- Same method name but different parameters.
- Achieved in the same class.

```
class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}

Calculator c = new Calculator();
System.out.println(c.add(2, 3)); // 5
System.out.println(c.add(2.5, 3.5)); // 6.0
System.out.println(c.add(2, 3, 4)); // 9
```

9. Practical Example

```
class Demo {
    int add(int a, int b) {
        return a + b;
    }
    void printMsg() {
        System.out.println("Welcome to Java!");
    }
}

public class Main {
    public static void main(String[] args) {
        Demo d = new Demo();
        int ans = d.add(7, 8);
        System.out.println("Sum = " + ans);
        d.printMsg();
    }
}
```

Output :
Sum = 15
Welcome to Java!

6. Types of Methods

(a) Static Method

- Belongs to the class.
- Can be called without creating object.
- Cannot access non-static data directly.

```
class MathUtils {
    static int add(int a, int b) {
        return a + b;
    }
}
```

```
int sum = MathUtils.add(10, 20);
```

(b) Instance Method

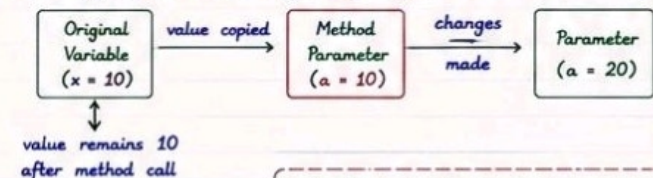
- Belongs to the object.
- Must be called using object reference.
- Can access both static and non-static data.

```
class Student {
    String name;
    void show() {
        System.out.println(name);
    }
}
```

```
Student s = new Student();
s.name = "Abhi";
s.show(); // Abhi
```

8. Pass-by-Value (In Java)

- Java always uses pass-by-value.
- A copy of the value is passed, original value is not changed.



★ 10. Interview Tip

Method Overloading Rules

- Must differ in number of parameters, OR
- Must differ in data type of parameters, OR
- Both.

Note : Return type alone cannot be used for overloading.



Good methods make good programs. Write once, use anywhere !





7. Arrays & Strings in Java

★ Key Point

Arrays store multiple values of same type.
Strings are classes in Java.



A. ARRAYS

1. What is an Array?

- An array is a collection of elements of the same data type stored in contiguous memory locations.

2. Declaration & Initialization

```
dataType[] arrayName = new dataType[size];
```

Example :

```
int[] marks = new int[5];
```

3. 1D Array

Example :

```
int[] arr = {10, 20, 30, 40, 50};
```

Index : 0 1 2 3 4

4. Array Traversal

```
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

arr.length gives total number of elements.

5. Enhanced for Loop

```
for (int num : arr) {
    System.out.println(num);
}
```

→ For each element in the array.

6. 2D Array

Example :

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

Access : matrix[1][2] = 6

6. == vs .equals()

== (Equality Operator)	.equals() (Method)
- Compares memory address (reference) - Returns true if both references point to same object.	- Compares content (actual value) - Returns true if contents are same, even if objects are different.

Example :

```
String a = "Hi";
```

```
String b = "Hi";
```

```
System.out.println(a == b); // true
```

```
String x = new String("Hi");
```

```
String y = new String("Hi");
```

```
System.out.println(x.equals(y)); // true
```

```
System.out.println(x == y); // false
```

Memory Representation (1D Array)

```
int[] arr = {10, 20, 30, 40, 50};
```

10	20	30	40	50
----	----	----	----	----

Index → 0 1 2 3 4

All elements are stored in contiguous memory locations.

B. STRINGS

1. What is String?

- In Java, String is a class that represents a sequence of characters.

2. String Creation

```
String s1 = "Hello"; // using string literal
```

```
String s2 = new String("Hello"); // using new keyword
```

```
char[] ch = {'J', 'a', 'v', 'a'};
```

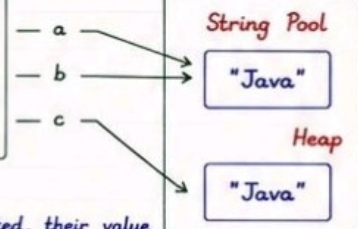
```
String s3 = new String(ch); // from char array
```

3. String Pool

```
String a = "Java";
```

```
String b = "Java";
```

```
String c = new String("Java");
```



4. String Immutability

- Strings are immutable. Once created, their value cannot be changed.

```
String s = "Abhi";
```

```
s = s + " Kumar"; // creates new object
```

5. Important String Methods

s.length() → returns length of string

s.charAt(i) → returns character at index i

s.substring(i, j) → returns substring from i to j-1

s.equals(str) → compares content

s.contains("hi") → returns true if "hi" exists

7. Practical Example

```
String name = "Java Programming";
```

```
System.out.println(name.length()); // 16
```

```
System.out.println(name.charAt(0)); // J
```

```
System.out.println(name.substring(0, 4)); // Java
```

```
System.out.println(name.contains("Pro")); // true
```

★ Remember

Arrays are fixed in size.

Strings are flexible and immutable.



★ Understand Arrays deeply. Master Strings completely. Code like a Pro! 😊



8. OOP Fundamentals in Java

★ Key Point

OOP helps us write clean, modular and maintainable code by modelling real-world things as objects.



1. What is OOP ?

- OOP stands for Object Oriented Programming.
- It is a programming paradigm that uses objects and classes to solve real-world problems.

2. Why Use OOP ?

- Code reusability
- Better organization
- Easy maintenance
- Modularity
- It represents real-world entities

4. Attributes (Fields)

Attributes are the data or properties of an object. These are declared inside the class.

```
class Student {
    int rollNo;      // attribute
    String name;     // attribute
    double marks;    // attribute
}
```

6. Creating Objects in Java

We create objects using the "new" keyword.

```
Student s1 = new Student(); // object creation
s1.display();               // method call
```

8. Types of Constructors

(a) Default Constructor

No parameters.
Provided by compiler if we don't create any.

```
class A {
    A() {
        System.out.println("Default");
    }
}
```

(b) Parameterized Constructor

Takes parameters to initialize the object.

```
class A {
    A(int x, String name) {
        System.out.println(x + " " + name);
    }
}
```

3. Class vs. Object

Class

- A class is a blueprint or template.
- It defines properties (attributes) and behaviors (methods).

Object

- An object is a real-world entity.
- It is an instance of a class.

Example :

Class → Car (blueprint)

Objects → BMW, Audi, Tesla (real cars)



5. Methods

Methods are the actions or behaviors of an object. They are defined inside the class.

```
class Student {
    void display() {
        System.out.println("Student Details");
    }
}
```

7. Constructors

Constructor is a special method used to initialize object. It has the same name as the class.

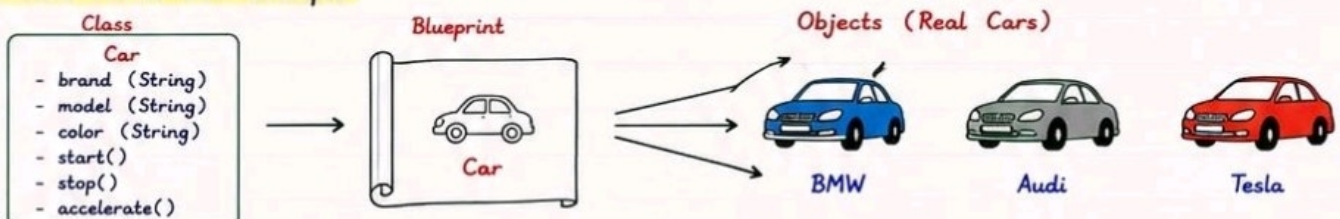
```
class Student {
    Student() {
        System.out.println("Constructor Called");
    }
}
```

9. this Keyword

The "this" keyword refers to the current object. It is used to resolve the naming conflict between attributes and parameters.

```
class Student {
    int rollNo;
    String name;
    Student(int rollNo, String name) {
        this.rollNo = rollNo; // current object's rollNo
        this.name = name;    // current object's name
    }
}
```

10. Real-World OOP Example



Think in Objects. Code in Classes. Build Great Programs !





9. Four Pillars of OOP in Java

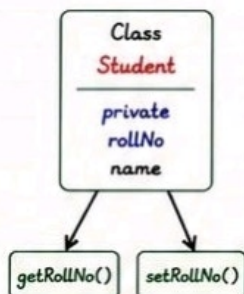
★ Key Point

The four pillars work together to build strong, flexible and scalable programs. 😊

1. Encapsulation

- Binding data (fields) and methods together in a single unit (class).
- Hides data from outside world.
- Achieved using private and accessors (getters & setters).

```
class Student {
    private int rollNo;
    private String name;
    // getter
    public int getRollNo() {
        return rollNo;
    }
    // setter
    public void setRollNo(int rollNo) {
        this.rollNo = rollNo;
    }
}
```



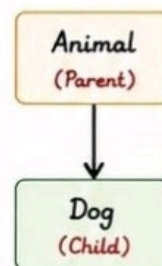
Real World: Bank account hides balance and allows access via methods.



2. Inheritance

- One class inherits properties and methods of another class.
- Promotes code reusability.
- Achieved using extends keyword.
- Parent → Child relationship.

```
class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Barking...");
    }
}
```



Real World: Dog inherits features of Animal and has its own behavior.



3. Polymorphism

- Ability of an object to take many forms.
- Same method name, different behavior.
- Two types:
 - ① Compile-time (Method Overloading)
 - ② Runtime (Method Overriding)

(a) Compile-time Method Overloading

```
class MathUtil {
    int add(int a, int b) {
        return a + b;
    }
    double add(double a, double b) {
        return a + b;
    }
}
```

(b) Runtime Method Overriding

```
class Animal {
    void sound() {
        System.out.println("Some sound");
    }
}
class Dog extends Animal {
    void sound() {
        System.out.println("Bark");
    }
}
```

Real World: Same command (sound()) behaves differently for different objects.



4. Abstraction

- Hiding implementation details and showing only essential features.
- Achieved using abstract classes or interfaces.
- Focus on what an object does, not how it does.

(a) Abstract Class

```
abstract class Shape {
    abstract void draw();
}
class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}
```

(b) Interface (Overview)

```
interface Vehicle {
    void start();
    void stop();
}
class Car implements Vehicle {
    public void start() {
        System.out.println("Car starts");
    }
    public void stop() {
        System.out.println("Car stops");
    }
}
```

Real World: You know what to do (draw, start) but not how it is done.

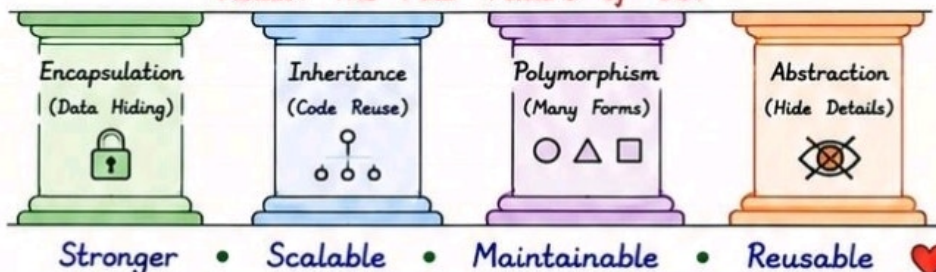


5. All Together

- Encapsulation protects data.
- Inheritance reuses code.
- Polymorphism adds flexibility.
- Abstraction reduces complexity.
- Together they make our programs powerful and easy to maintain. 😊



Visual: The Four Pillars of OOP





10.

Constructors, Inheritance & Polymorphism in Java

★ Key Point

Constructors create objects.
Inheritance reuses & extends.
Polymorphism gives flexibility.



1. Constructors in Java

- A constructor is a special method used to initialize objects.
- Constructor name must be same as the class name and has no return type.

(a) Default Constructor (no parameter)

```
class Student {
    int rollNo;
    Student() {
        rollNo = 0;
    }
}
```

Called automatically if no constructor is defined.

(b) Parameterized Constructor

```
class Student {
    int rollNo; String name;
    Student(int r, String n) {
        rollNo = r;
        name = n;
    }
}
```

Takes parameters and initializes object.

(c) Constructor Overloading

```
class Student {
    Student() {}
    Student(int r) { rollNo = r; }
    Student(int r, String n) {
        rollNo = r; name = n;
    }
}
```

Multiple constructors with different parameter lists.

(d) Constructor Chaining

```
class Student {
    int rollNo; String name;
    Student() { this(0, "Unknown"); }
    Student(int r) { this(r, "Unknown"); }
    Student(int r, String n) {
        rollNo = r; name = n;
    }
}
```

this() is used to invoke another constructor in the same class.

5. this() and super()

- this() → calls another constructor in the same class.
- super() → calls constructor of the parent class.

Using this()

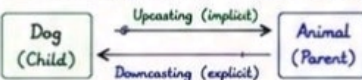
```
class A {
    A() { this(10); }
    A(int x) { System.out.println(x); }
}
```

Using super()

```
class A {
    A() { System.out.println("A"); }
}
class B extends A {
    B() { super(); System.out.println("B"); }
}
```

6. Upcasting & Downcasting

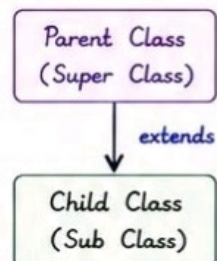
- Upcasting : Child → Parent (implicit)
- Downcasting : Parent → Child (explicit)



```
Animal a = new Dog(); // Upcasting
Dog d = (Dog) a; // Downcasting
d.bark();
```

2. Inheritance in Java

- One class inherits properties and methods of another class.
- Achieved using 'extends' keyword.
- Parent class (super class) → Child class (sub class).



Example :

```
class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Barking...");
    }
}
```

Dog inherits eat() from Animal and has its own method bark().

3. Method Overriding (Runtime Polymorphism)

- Child class provides its own implementation of a method already defined in parent class.
- Achieved by same method name and signature.
- Helps in runtime polymorphism.

```
class Animal {
    void sound() {
        System.out.println("Some sound");
    }
}
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}
```

Output :

```
Animal a = new Dog();
a.sound(); // Bark
```

At runtime, JVM calls Dog's sound() method.

4. Polymorphism

- Ability of an object to take many forms.
- In Java, achieved through method overriding.
- Two types → Compile-time (overloading) & Runtime (overriding).

Types of Polymorphism

Compile-time Polymorphism
(Method Overloading)
Resolved during compile time.

Runtime Polymorphism
(Method Overriding)
Resolved during runtime.

★ Interview Box : Overloading vs Overriding

Overloading (Compile-time)	Overriding (Runtime)
1. Same method name	Same method name
2. Different parameter list	Same parameter list
3. May change return type	Same return type
4. Happens in same class	Happens in parent-child classes
5. Resolved at compile time	Resolved at runtime (JVM)





11.

Interfaces, Abstract Classes & Access Modifiers in Java

★ Key Point

Abstraction helps us focus on **WHAT** an object does, not **HOW** it does.

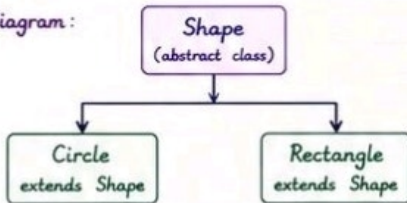


1. Abstract Class

- A class declared with "abstract" keyword.
- Can have abstract and non-abstract methods.
- Cannot create object of abstract class.
- Used to provide a common base.

```
abstract class Shape {
    abstract void draw(); // abstract method
    void display() {       // concrete method
        System.out.println("Shape class");
    }
}
```

Diagram:



2. Abstract Methods

- Declared without body.
- Must be implemented by subclass.
- Only in abstract class.

```
abstract class Animal {
    abstract void sound(); // no body
}
```

Any class that has at least one abstract method must be abstract.

3. Interface

- Blueprint of a class.
- Contains only abstract methods (before Java 8).
- Fields are public, static & final by default.
- Achieved using "interface" keyword.

```
interface Vehicle {
    void start();
    void stop();
}
```

4. implements Keyword

- Used by a class to implement an interface.
- Class must implement all methods of interface.
- Supports multiple inheritance.

```
class Car implements Vehicle {
    public void start() { System.out.println("Car Start"); }
    public void stop() { System.out.println("Car Stop"); }
}
```

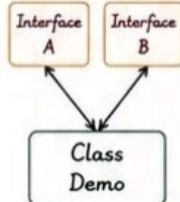
A class can implement multiple interfaces.

5. Multiple Inheritance through Interfaces

- A class can implement multiple interfaces.

```
interface A { void showA(); }
interface B { void showB(); }

class Demo implements A, B {
    public void showA() {
        System.out.println("A");
    }
    public void showB() {
        System.out.println("B");
    }
}
```



6. Abstract Class vs Interface

Feature	Abstract Class	Interface
Keyword	abstract class	interface
Methods	Abstract + Concrete	Only Abstract (before Java 8)
Variables	Can have any type	public static final
Constructor	Yes	No
Inheritance	Single	Multiple
Access Modifiers	Any	public (methods are public)
When to use?	When we have "is-a" relationship	When we need capability (can do)

7. Access Modifiers

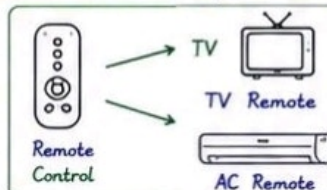
- Used to control visibility of classes, methods and variables.
- There are 4 types of access modifiers.

Modifier	Same Class	Same Package	Sub Class (Different Package)	Outside Package (Non-Subclass)
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default (no modifier)	✓	✓	✗	✗
private	✓	✗	✗	✗

Example

```
class Test {
    public int a; // accessible everywhere
    protected int b; // same pkg + subclass
    int c; // default: same package
    private int d; // only in this class
}
```

Real-World Example (Abstraction)



We know **WHAT** buttons do, not **HOW** the devices work internally.

8. Interview Tip - When to use Interface vs Abstract Class?

Use Interface when:

- You want to define a capability.
- Future requirement is not known.
- You need multiple inheritance.
- Example: Runnable, Serializable, Comparator

Use Abstract Class when:

- You want to share common code.
- You have "is-a" relationship.
- You want to define some default behavior.
- Example: AbstractList, AbstractMap



Abstraction reduces complexity. Good design builds better code!





12. Exception Handling in Java

★ Key Point

Exceptions are unexpected events. Handling them makes your program robust and user-friendly. 😊

1. What is an Exception?

- An exception is an event that occurs during the execution of a program and disrupts the normal flow of instructions.
- Example: Divide by zero, File not found, Null pointer etc.

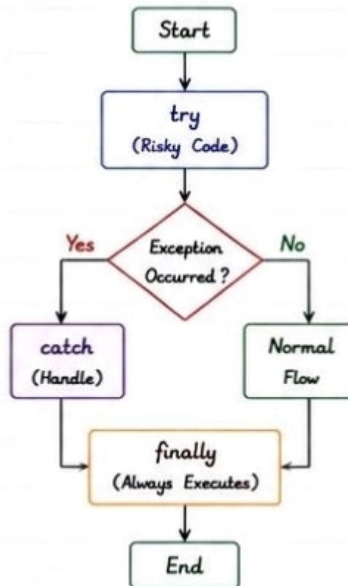
2. Why Exception Handling?

- Maintains normal flow of the application.
- Provides meaningful error messages.
- Improves code reliability.
- Prevent abnormal termination.

3. Keywords in Exception Handling

- try** → Encloses risky code.
- catch** → Catches and handles exception.
- finally** → Always executes (cleanup code).
- throw** → Manually throws an exception.
- throws** → Declares exception (passed to caller).

4. try - catch - finally Flow



5. Basic Example

```

public class Demo {
    public static void main(String[] args) {
        try {
            int a = 10, b = 0;
            int c = a / b; // risky code
            System.out.println("Result = " + c);
        }
        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero!");
        }
        finally {
            System.out.println("Finally block executed.");
        }
        System.out.println("Program continues...");
    }
}
  
```

Output:
Cannot divide by zero!
Finally block executed.
Program continues...

6. Multiple catch Blocks

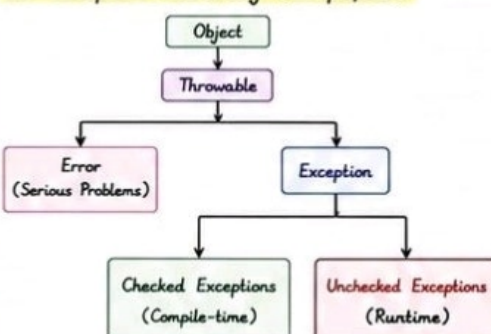
```

try {
    int a[] = new int[5];
    a[10] = 50 / 0; // two possible exceptions
}
catch (ArithmeticException e) {
    System.out.println("Arithmetic Exception");
}
catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Array Index Out Of Bounds");
}
catch (Exception e) {
    System.out.println("Some Other Exception");
}
  
```

8. Common Built-in Exceptions

- **ArithmeticException** → Divide by zero
- **NullPointerException** → Using null object
- **ArrayIndexOutOfBoundsException** → Invalid array index
- **NumberFormatException** → Invalid number format
- **IOException** → Input/Output failure
- **ClassNotFoundException** → Class not found

10. Exception Hierarchy (Simplified)



7. Checked vs Unchecked Exceptions

Feature	Checked Exceptions	Unchecked Exceptions
Checked at	Compile-time	Runtime
Examples	IOException, SQLException	ArithmeticException, NullPointerException
Handling	Must be handled (using try-catch or throws)	Not mandatory (but recommended)
Inheritance	Extend Exception class	Extend RuntimeException class

9. Custom Exception Example

```

class MyException extends Exception {
    MyException(String msg) {
        super(msg);
    }
}
  
```

Output:
Age must be 18 or more

```

class Test {
    static void validate(int age) throws MyException {
        if (age < 18)
            throw new MyException("Age must be 18 or more");
    }
    public static void main(String[] args) {
        try { validate(16); }
        catch (MyException e) {
            System.out.println(e.getMessage());
        }
    }
}
  
```

★ Interview Tip

- throw** → Used to manually throw an exception.
- throws** → Declares that a method may throw an exception.
- finally** → Always executes whether exception occurs or not.

11. throw vs throws

Aspect	throw	throws
Purpose	Used to throw an exception explicitly.	Used to declare exception.
Where used	Inside method body.	In method signature.
No. of exceptions	Only one exception at a time.	Multiple exceptions can be declared.
Example	<code>throw new IOException();</code>	<code>void read() throws IOException;</code>





13.

Java Collections Framework

★ Key Point

Collections help us store, manipulate and retrieve groups of objects efficiently.



1. Why Collections ?

- Dynamic in size.
- Increase code reusability.
- Provide powerful data structures.
- Better performance & easy to use.
- Part of java.util package.

2. Core Interfaces in Collections

Interface	Description
List	Ordered collection (allows duplicates).
Set	Unordered collection (no duplicates).
Queue	Follows FIFO order.
Map	Key-Value pair (unique keys).

3. Common Methods (used in most collections)

add(e)	→ Adds an element.
remove(e)	→ Removes an element.
contains(e)	→ Checks if element exists.
size()	→ Returns number of elements.
isEmpty()	→ Checks if collection is empty.
clear()	→ Removes all elements.

5. ArrayList vs LinkedList

Feature	ArrayList	LinkedList
Implementation	Resizable Array	Doubly Linked List
Order	Maintains insertion order	Maintains insertion order
Duplicates	Allowed	Allowed
Random Access	Yes (get/set by index)	No (sequential access)
Performance	Faster for retrieval	Faster for insertion/deletion
Use when	More reads, less writes	More writes, less reads

ArrayList Example

```
List<String> list = new ArrayList<>();
list.add("Apple");
list.add("Banana");
System.out.println(list); // [Apple, Banana]
```

LinkedList Example

```
List<Integer> list = new LinkedList<>();
list.add(10);
list.addFirst(5);
list.addLast(20);
System.out.println(list); // [5, 10, 20]
```

7. HashMap vs HashSet

Feature	HashMap	HashSet
Stores	Key-Value pairs	Only values
Null	One null key & many null values	One null value
Duplicates	Keys - No, Values - Yes	No duplicates
Order	No order	No order
Example	<Integer, String>	<Integer>

8. Iterator (to traverse collections)

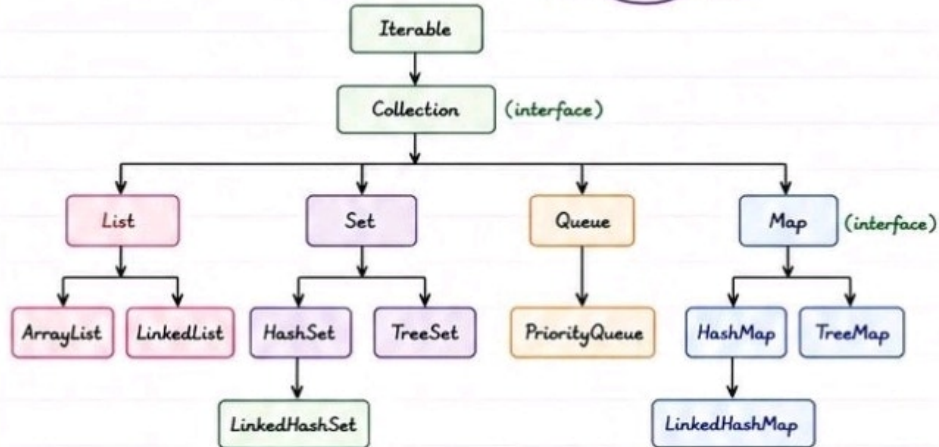
```
List<String> list = new ArrayList<>();
list.add("A"); list.add("B");
list.add("C");

Iterator<String> it = list.iterator();
while(it.hasNext()) {
    System.out.println(it.next());
}
```

Iterator is used to access elements one by one without knowing internal implementation.



4. Collections Hierarchy (Overview)



★ Note: All these classes are in java.util package.

6. HashSet vs TreeSet

Feature	HashSet	TreeSet
Ordering	No order	Sorted (natural order)
Duplicates	Does not allow	Does not allow
Null Elements	Allows one null	Does not allow null
Performance	Faster	Slower (due to sorting)
Implementation	Hash Table	Red-Black Tree

HashSet Example

```
Set<Integer> set = new HashSet<>();
set.add(1);
set.add(2);
set.add(2); // duplicate ignored
System.out.println(set); // [1, 2]
```

TreeSet Example

```
Set<Integer> set = new TreeSet<>();
set.add(3);
set.add(1);
set.add(2);
System.out.println(set); // [1, 2, 3]
```

HashMap Example

```
Map<Integer, String> map = new HashMap<>();
map.put(1, "One");
map.put(2, "Two");
System.out.println(map.get(1)); // One
System.out.println(map.keySet()); // [1, 2]
```

HashSet Example

```
Set<String> set = new HashSet<>();
set.add("A");
set.add("B");
set.add("A"); // duplicate ignored
System.out.println(set); // [A, B]
```

9. Time Complexity (Big-O)

Operation	ArrayList	LinkedList	HashSet	TreeSet	HashMap
add(e)	O(1) *	O(1)	O(1)	O(log n)	O(1)
remove(e)	O(n)	O(1)	O(1)	O(log n)	O(1)
contains(e)	O(n)	O(n)	O(1)	O(log n)	O(1)
get(index)	O(1)	O(n)	—	—	O(1) **

* Amortized O(1) ** get(key)



Choose the right Collection for the right use case. It makes your code faster and cleaner!





14.

Multithreading + Modern Java Basics

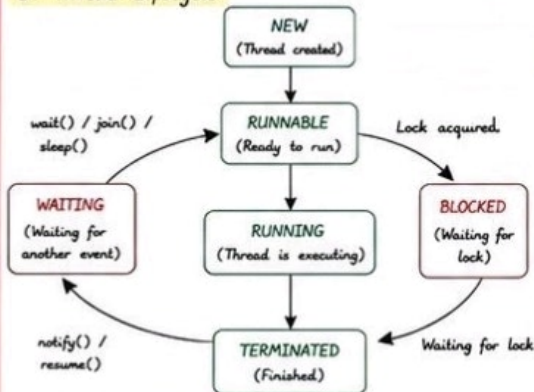
1. What is a Thread?

- A thread is the smallest unit of execution inside a process.
- Multiple threads share memory but have their own stack.

2. Process vs Thread

Process	Thread
Has its own memory	Shares memory with other threads
Heavy weight	Light weight
Slower to create	Faster to create
High context switch cost	Low context switch cost

3. Thread Lifecycle



4. Creating Threads using Thread class

```

class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread: " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start(); // starts a new thread
    }
}
  
```

Note:
Always use `start()` to run a thread, not `run()`.



5. Creating Threads using Runnable interface

```

class MyTask implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Runnable: " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Runnable r = new MyTask();
        Thread t = new Thread(r);
        t.start(); // starts a new thread
    }
}
  
```

Why Runnable?

- Supports multiple inheritance
- Decouples task from thread
- Preferred way to create threads



6. start() vs run()

start()	run()
Creates a new thread and calls run().	Does NOT create a new thread.
Executes in a separate call stack.	Runs in the current thread.
Can run concurrently with other threads.	No concurrency (acts like a normal method call).

Example

```

Thread t = new MyThread();
t.start(); // new thread
t.run(); // same main thread
  
```

7. Synchronization (Overview)

- Multiple threads can access shared data, causing inconsistent results.
- synchronization keyword ensures only one thread can access the block/method at a time.

```

class Counter {
    private int count = 0;
    public synchronized void increment() {
        count++;
    }
}
  
```

8. Lambda Expressions

- Introduced in Java 8.
- Used to represent a function.
- Makes code shorter and clean.

Example

```

(int a, int b) -> a + b; // lambda
Runnable r = () -> System.out.println("Hi");
  
```

9. Functional Interfaces

- An interface with exactly one abstract method.
- Can be used with lambda expressions.

Example

```

@FunctionalInterface
interface MyFunc {
    void show(String msg); // single abstract method
}

MyFunc f = (msg) -> System.out.println(msg);
f.show("Hello Java 8!");
  
```

★ Key Point

Threads help us run multiple tasks together. Modern features make Java clean, concise and powerful. 😊

10. Stream API (Overview)

- Introduced in Java 8.
- Used to process collections in a declarative way.
- Supports operations like filter, map, sort, collect, etc.

Example

```

List<Integer> list = Arrays.asList(1, 2, 3, 4, 5, 6);
List<Integer> evenSquares = list.stream()
    .filter(n -> n % 2 == 0) // 2, 4, 6
    .map(n -> n * n) // 4, 16, 36
    .collect(Collectors.toList());

System.out.println(evenSquares); // [4, 16, 36]
  
```

11. Small Practical Examples

(a) Print numbers using threads

```

class PrintTask implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread.currentThread().getName() + " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t1 = new Thread(new PrintTask(), "T1");
        Thread t2 = new Thread(new PrintTask(), "T2");
        t1.start(); t2.start();
    }
}
  
```

(b) Using lambda with forEach

```

List<String> names = Arrays.asList("Ari", "Bob", "Cat");
names.forEach(n -> System.out.println(n));
  
```

(c) Using Stream to get squares of odd numbers

```

List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
nums.stream()
    .filter(n -> n % 2 != 0) // odd
    .map(n -> n * n) // square
    .forEach(System.out::println);
  
```

★ Interview Tip

Why `start()` creates a new thread but `run()` does not?

- ✓ `start()` tells JVM to create a new thread, allocate a new call stack, and then call `run()` on that new thread.
- ✓ `run()` is just a normal method call. When you call it directly, it executes in the current thread's call stack.





15.

Java Interview Cheat Sheet & Final Revision

★ Key Point

Revision is the key to confidence. Keep practicing and keep building!



1. JDK vs JRE vs JVM

JDK	Java Development Kit. It contains JRE + compiler + tools (javac, java, javadoc).
JRE	Java Runtime Environment. It contains JVM + core libraries.
JVM	Java Virtual Machine. It executes bytecode and provides runtime environment.

2. Primitive Data Types

- byte - 1 byte
- short - 2 bytes
- int - 4 bytes
- long - 8 bytes
- float - 4 bytes
- double - 8 bytes
- char - 2 bytes (Unicode)
- boolean - true/false

Numeric Types

3. == vs .equals()

== → compares references (memory address)
.equals() → compares content (value)
Example
 String a = new String("Hi");
 String b = new String("Hi");
 System.out.println(a == b); // false
 System.out.println(a.equals(b)); // true

4. String Immutability & String Pool

- String objects are immutable.
- Modifying creates a new object.
- String literals are stored in String Pool.



String s1 = "Hello"; // from pool
 String s2 = "Hello"; // same reference
 String s3 = new String("Hello"); // new object

5. final vs finally vs finalize

final	Keyword. Variable → constant Method → cannot override Class → cannot inherit
finally	Block that always executes after try-catch.
finalize	Method of Object class. Called by GC before destroying an object. (Deprecated)

6. Overloading vs Overriding

Overloading (Compile-time)

- Same method name
- Different parameters
- In same class

Overriding (Runtime)

- Same method name
- Same parameters
- In parent-child classes
- Achieves runtime polymorphism

7. Abstract Class vs Interface

Abstract Class	Interface
Can have abstract & concrete methods	Only abstract methods (before Java 8)
Can have fields	Only public static final constants
Single inheritance	Multiple inheritance possible
Uses "extends"	Uses "implements"

8. ArrayList vs LinkedList

ArrayList	LinkedList
Resizable array	Doubly linked list
Fast random access (O(1))	Slow random access (O(n))
Fast for read	Fast for insert/delete
Uses more memory	Uses less memory

9. HashMap vs HashSet

HashMap	HashSet
Stores key-value pairs	Stores unique values
Keys → unique	No duplicate values
Allows one null key & multiple null values	Allows one null value
Uses HashMap internally	Uses HashMap internally

10. Checked vs Unchecked Exceptions

Checked Exceptions	Unchecked Exceptions
Checked at compile-time	Checked at runtime
Must be handled (using try-catch or throws)	Not mandatory to handle
Extend Exception class (except RuntimeException)	Extend RuntimeException class
Example: IOException, SQLException	Example: NullPointerException, ArithmeticException

11. static Keyword

- Belongs to class, not to object.
- Can access without creating object.
- Shared among all objects.

Example

```

class Test {
    static int count = 0;
    static void show() {
        System.out.println(count);
    }
}
    
```

12. this vs super

this

- Refers to current object.
- Used to access instance variables, methods.
- Used to call current class constructor.

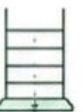
super

- Refers to parent class object.
- Used to access parent class members.
- Used to call parent constructor.

13. Stack vs Heap Memory

Stack (Primitive / Reference)

- Stores method calls, local variables.
- Faster access.
- Size is smaller.



Heap (Objects)

- Stores objects and arrays.
- Created using 'new' keyword.
- Size is larger.

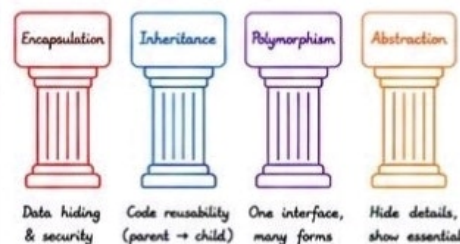


14. Common Java Keywords

abstract assert boolean break byte case catch char
 class const* continue default do double else enum
 extends final finally float for if implements import
 instanceof int interface long native new package
 private protected public return short static strictfp
 super switch synchronized this throw throws transient
 try void volatile while

*const is reserved (not used)

15. Four Pillars of OOP



16. Final Java Learning Path



Practice daily.
 Build projects.
 Never stop learning!